



UNIVERSIDAD DE ALMERÍA

Grado en Ingeniería Informática

Introducción a la Programación

2026-2027



Tema 1. Fundamentos de Programación

➤ Programa - Entrada por consola

Implementación de un algoritmo en un lenguaje de programación



Tema 1. Fundamentos de Programación

➤ Programa - Entrada por consola

Implementación de un algoritmo en un lenguaje de programación

Ejemplo: Programa en Java

```
3 public class Circunferencia {  
4  
5 public static void main(String[] args) {  
6     double radio = 4.5;  
7     double longitud = 2 * Math.PI * radio;  
8     double area = Math.PI * Math.pow(radio, 2.0);  
9  
10    System.out.println("Radio = " + radio);  
11    System.out.println();  
12    System.out.printf("Longitud de la circunferencia = %.4f", longitud);  
13    System.out.println();  
14    System.out.printf("Area de la circunferencia = %.4f", area);  
15    System.out.println();  
16 }  
17 }  
18
```

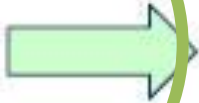
El dato de entrada al programa, el decir, el valor del radio, se encuentra **escrito directamente en el código fuente** como un **literal (4.5)**. Se dice que el valor está *"hardcoded"*

Tema 1. Fundamentos de Programación

➤ Programa - Entrada por consola

Ahora queremos que cada vez que se ejecute el programa, el **usuario introduzca el valor por teclado**

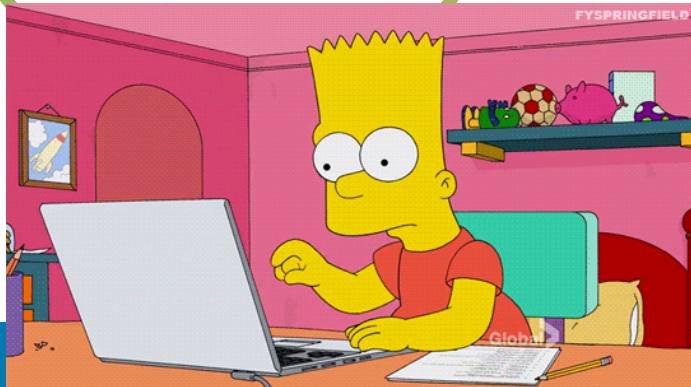
Datos de
entrada



Programa
Instrucciones



Datos de
salida



Tema 1. Fundamentos de Programación



Programa con Entrada desde la consola

```
import java.util.Scanner;

public class CircunferenciaEntrada {

    public static void main(String[] args) {

        Scanner entrada = new Scanner(System.in);
        // El usuario introduce el radio
        System.out.print("Introduzca el radio: ");
        double radio = entrada.nextDouble();

        double longitud = 2.0 * Math.PI * radio;
        double area = Math.PI * Math.pow(radio, 2.0);

        System.out.println("Radio = " + radio);
        System.out.println();
        System.out.printf("Perimetro de la circunferencia = %.3f", longitud);
        System.out.println();
        System.out.printf("Area de la circunferencia = %.3f", area);
        System.out.println();
        entrada.close();
    }
}
```

Tema 1. Fundamentos de Programación

➤ Scanner

Lectura de tipos de datos numéricos:

- `nextInt()`: Lee un número entero (`int`).
- `nextLong()`: Lee un número entero largo (`long`).
- `nextFloat()`: Lee un número en coma flotante (`float`).
- `nextDouble()`: Lee un número en coma flotante de doble precisión (`double`).
- `nextByte()`: Lee un valor de tipo `byte` (`byte`).
- `nextShort()`: Lee un número corto (`short`).

Lectura de cadenas:

- `next()`: Lee la siguiente palabra (token) de la entrada.
- `nextLine()`: Lee toda una línea de entrada.

Tema 1. Fundamentos de Programación

- Algoritmos y programas
- Introducción a la programación en Java
- Datos y tipos de datos
- Tipos de datos primitivos en Java
- Variables
- Expresiones y sentencias
- Programas
- Estructuras de control

Tema 1. Fundamentos de Programación



Estructuras de control

- ☐ Programación estructurada
- ☐ Estructuras selectivas/condicionales
- ☐ Estructuras repetitivas/iterativas

Teorema de Böhm y Jacopini (1966):

Cualquier programa de ordenador puede diseñarse e implementarse utilizando únicamente las tres construcciones estructuradas (secuencia, selección e iteración; esto es, sin sentencias goto).

En programación estructurada sólo se emplean tres construcciones:

☐ Secuencia

Conjunto de sentencias que se ejecutan **en orden**

Ejemplos: Sentencias de asignación (=).

☐ Selección

Elige **qué sentencias se ejecutan** en función de una condición.

Ejemplos: Estructuras de control condicional **if ... else** y **switch**

☐ Iteración

Las estructuras de control repetitivas **repiten conjuntos de instrucciones**.

Ejemplos: Bucles **while**, **do...while** y **for**.

Tema 1. Fundamentos de Programación



Estructuras de control

- ☐ Programación estructurada



- ☐ Estructuras selectivas/condicionales

- ☐ Estructuras repetitivas/iterativas

Tema 1. Fundamentos de Programación



Estructuras de control selectivas

Por defecto, las instrucciones de un programa se ejecutan **secuencialmente**

Sin embargo, al describir la resolución de un problema, es normal que tengamos que tener en cuenta condiciones que influyen sobre la secuencia de pasos que hay que dar para resolver el problema.

Las estructuras de control condicionales o selectivas nos permiten decidir qué ejecutar y qué no en un programa.

Estructuras de control selectivas

Sintaxis Selectiva Simple

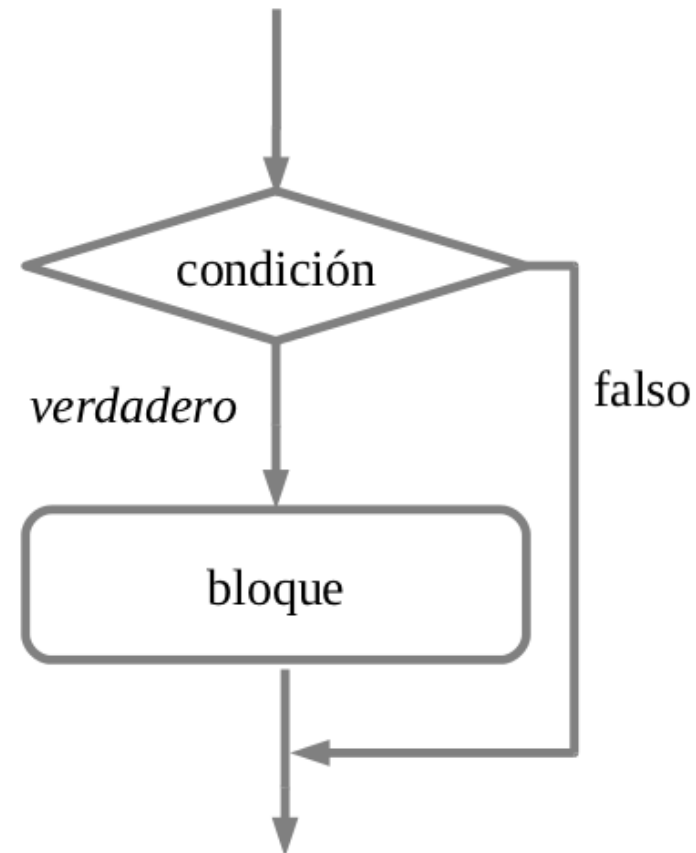
```
if (condición)  
    sentencia;
```

```
if (condición) {  
    bloque  
}
```

donde ***bloque*** representa un bloque de instrucciones.

Bloque de instrucciones:

Secuencia de instrucciones encerradas entre dos llaves {...}

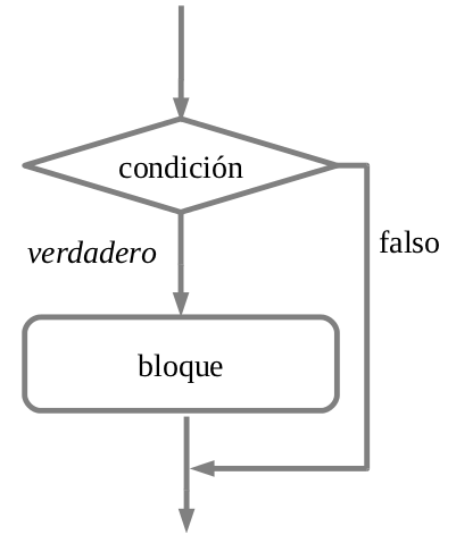


Estructuras de control selectivas

Ejemplos de uso selectiva simple:

```
if (nota >= 5)  
    System.out.println("APROBADO");
```

```
if (nota >= 5) {  
    System.out.println("APROBADO");  
    System.out.println("Recupera todo");  
}
```

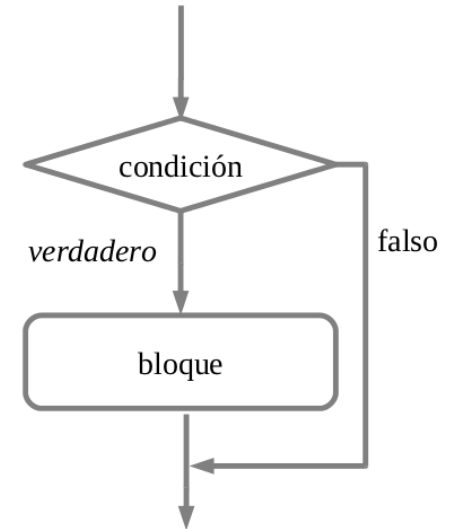


Estructuras de control selectivas

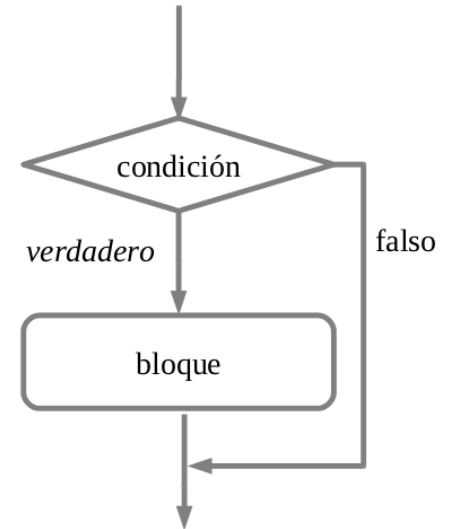
```

1 package ejemplosClaseT1;
2
3 import java.util.Scanner; //importamos esta libreria con utilidades para leer
4
5 public class NumeroPositivo {
6
7     public static void main(String[] args) {
8
9         int numero; // NO ponemos tildes !!
10
11         // Para leer desde teclado declaramos e inicializamos esta variable llamada entrada
12         Scanner entrada = new Scanner(System.in);
13
14         System.out.println("Introduzca un numero: ");
15
16         numero = entrada.nextInt(); //lee un int desde teclado y lo guarda en la variable numero
17
18         if (numero < 0 ) {
19             System.out.println("Este numero es negativo: " + numero);
20             numero = numero * -1;
21         }
22
23         System.out.println("El valor absoluto del numero es: " + numero);
24     }
25
26 }

```



Estructuras de control selectivas



```

1 package ejemplosClaseT1;
2
3 import java.util.Scanner; //importamos esta libreria con utilidades para leer
4
5 public class NumeroPositivo {
6
7     public static void main(String[] args) {
8
9         int numero; // NO ponemos tildes !!
10
11         // Para leer desde teclado declaramos e inicializamos esta variable llamada entrada
12         Scanner entrada = new Scanner(System.in);
13
14         System.out.println("Introduzca un numero: ");
15
16         numero = entrada.nextInt(); //lee un int desde teclado y lo guarda en la variable numero
17
18         → if (numero < 0 ) {
19             System.out.println("Este numero es negativo: " + numero);
20             numero = numero * -1;
21         }
22
23         → System.out.println("El valor absoluto del numero es: " + numero);
24     }
25
26 }
27

```

Introduzca un numero:

6

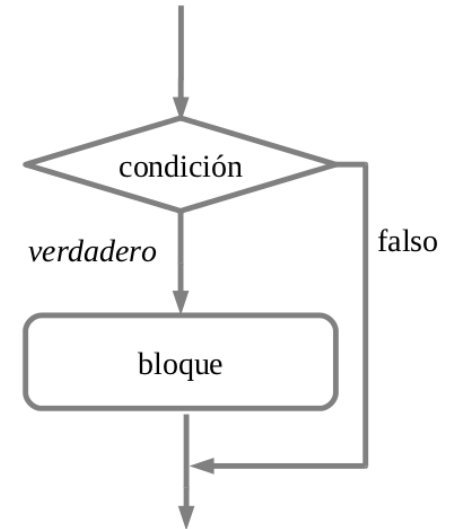
El valor absoluto del numero es: 6

Estructuras de control selectivas

```

1 package ejemplosClaseT1;
2
3 import java.util.Scanner; //importamos esta libreria con utilidades para leer
4
5 public class NumeroPositivo {
6
7     public static void main(String[] args) {
8
9         int numero; // NO ponemos tildes !!
10
11         // Para leer desde teclado declaramos e inicializamos esta variable llamada entrada
12         Scanner entrada = new Scanner(System.in);
13
14         System.out.println("Introduzca un numero: ");
15
16         numero = entrada.nextInt(); //lee un int desde teclado y lo guarda en la variable numero
17
18         → if (numero < 0 ) {
19             → System.out.println("Este numero es negativo: " + numero);
20             → numero = numero * -1;
21         }
22
23         → System.out.println("El valor absoluto del numero es: " + numero);
24     }
25
26 }
27

```



```

Introduzca un numero:
-8
Este numero es negativo: -8
El valor absoluto del numero es: 8

```

Estructuras de control selectivas

Consideraciones acerca del uso de la sentencia **if**:

- **No olvidar los paréntesis** al poner la condición del **if**, es un error sintáctico (los paréntesis son necesarios)
- **No hay que confundir el operador de comparación == con el operador de asignación =**
- Los operadores de comparación ==, !=, <= y >=, han de escribirse **sin espacios**.
- == y <= **no son operadores válidos en Java**.
- El fragmento de código afectado por la condición del **if** debe sangrarse o indentarse para que visualmente se interprete correctamente el ámbito de la sentencia **if**:

```
if (condición) {  
    // Aquí se incluye el código  
    // que ha de ejecutarse  
    // cuando se cumple la condición del if  
}
```

Estructuras de control selectivas

➤ Aunque el uso de llaves no sea obligatorio cuando el **if** sólo afecta a una sentencia, es recomendable ponerlas siempre para delimitar explícitamente el ámbito de la sentencia **if**.

➤ **Error común:**

```
if (condición); ← mal el punto y coma
    sentencia;
```

// es interpretado como

```
if (condición)
```

```
    ;           // Sentencia vacía
```

```
sentencia;
```

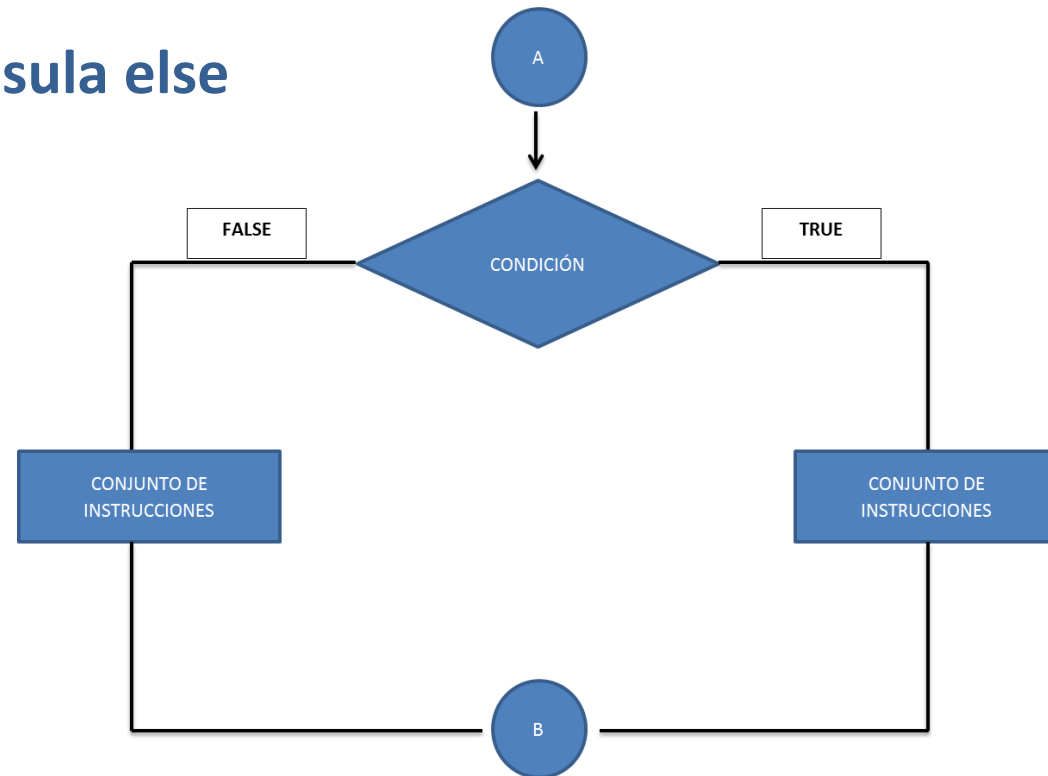
!!!La **sentencia** siempre se ejecutaría!!!

Estructuras de control selectivas

Sintaxis Selectiva Doble, Cláusula else

```
if (condición)  
    sentencia1;  
else  
    sentencia2;
```

```
if (condición) {  
    bloque1;  
} else {  
    bloque2;  
}
```

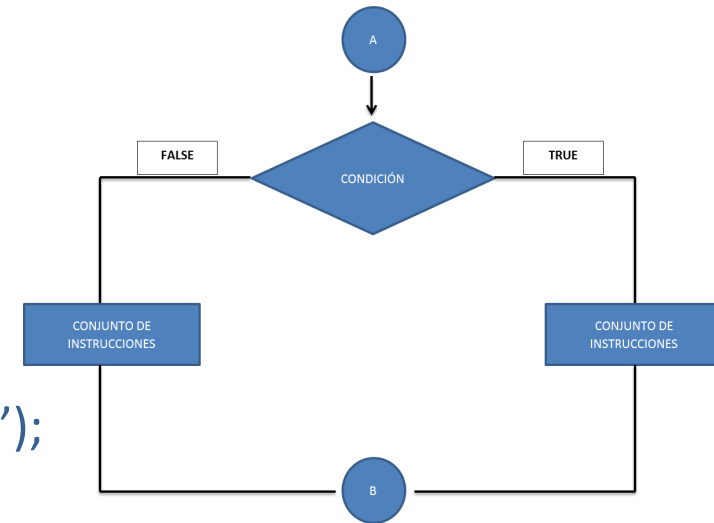


Estructuras de control selectivas

Ejemplos de uso selectiva doble

```
if (nota >= 5)
    System.out.println("APROBADO");
else
    System.out.println("SUSPENSO");
```

```
if (nota >= 5) {
    System.out.println("APROBADO");
    System.out.println("Recupera todo");
} else {
    System.out.println("SUSPENSO");
    System.out.println("No recupera");
}
```

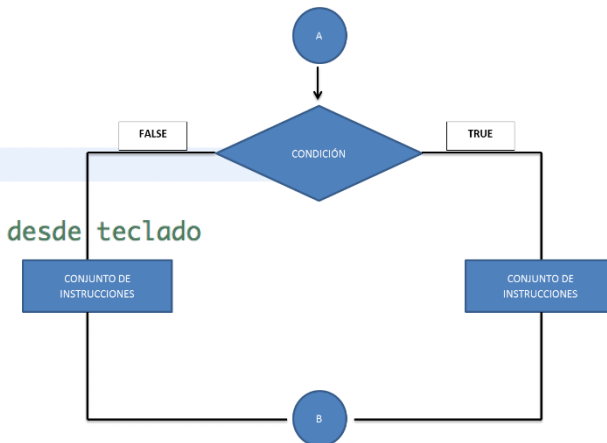


Estructuras de control selectivas

```

1 package ejemplosClaseT1;
2
3 import java.util.Scanner; //importamos esta libreria con utilidades para leer desde teclado
4
5 public class NumeroCorrecto {
6
7     public static void main(String[] args) {
8
9         int numero; // NO ponemos tildes !!
10
11         // Para leer desde teclado declaramos e inicializamos esta variable llamada entrada
12         Scanner entrada = new Scanner(System.in);
13
14         System.out.println("Introduzca un numero: ");
15
16         numero = entrada.nextInt(); //lee un int desde teclado y lo guarda en la variable numero
17
18         if (numero < 0 ) {
19             System.out.println("Este numero es negativo: " + numero);
20             numero = numero * -1;
21         }
22         else {
23             System.out.println("Este numero es positivo: " + numero);
24             System.out.println("Correcto! ");
25         }
26
27         System.out.println("El valor absoluto del numero es: " + numero);
28
29     }
30
31 }
32

```

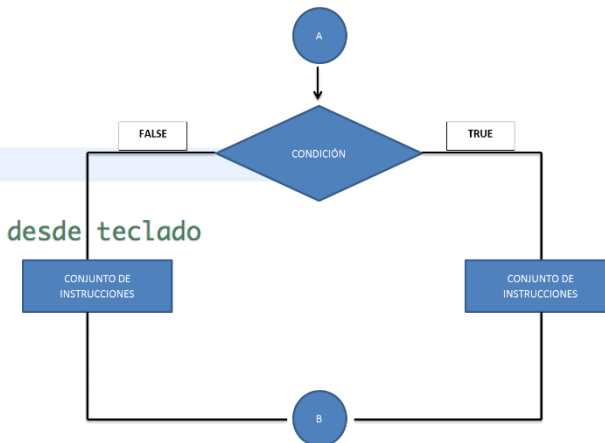


Estructuras de control selectivas

```

1 package ejemplosClaseT1;
2
3 import java.util.Scanner; //importamos esta libreria con utilidades para leer desde teclado
4
5 public class NumeroCorrecto {
6
7     public static void main(String[] args) {
8
9         int numero; // NO ponemos tildes !!
10
11         // Para leer desde teclado declaramos e inicializamos esta variable llamada entrada
12         Scanner entrada = new Scanner(System.in);
13
14         System.out.println("Introduzca un numero: ");
15
16         numero = entrada.nextInt(); //lee un int desde teclado y lo guarda en la variable numero
17
18         if (numero < 0 ) {
19             System.out.println("Este numero es negativo: " + numero);
20             numero = numero * -1;
21         }
22         else {
23             System.out.println("Este numero es positivo: " + numero);
24             System.out.println("Correcto! ");
25         }
26
27         System.out.println("El valor absoluto del numero es: " + numero);
28     }
29 }
30
31 }
32

```



```

Introduzca un numero:
43
Este numero es positivo: 43
Correcto!
El valor absoluto del numero es: 43

```

Estructuras de control selectivas

Ejemplos de uso selectiva con operadores relacionales

```
if (nota < 0 || nota > 10)
    System.out.println("NOTA NO VALIDA");
else
    System.out.println("VALIDA");
```

Estructuras de control selectivas

Ejemplos de uso selectiva con operadores relacionales

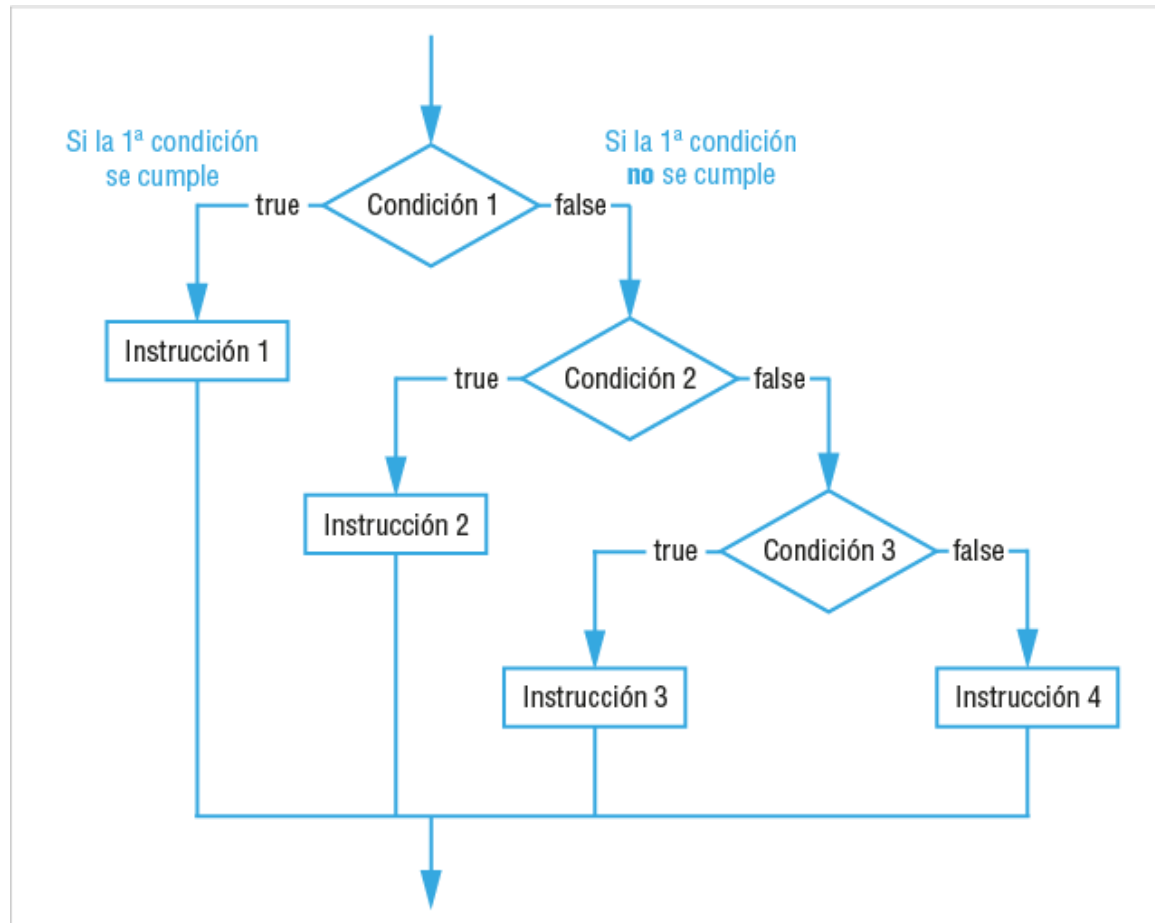
```
if (nota >= 0 && nota <= 10) {  
    System.out.println("VALIDA");  
} else {  
    System.out.println("NOTA NO VALIDA");  
}
```

Estructuras de control selectivas

Sintaxis Selectiva Múltiple if ... else if ...

```
if (condición1)
    sentencia1;
else if (condición2)
    sentencia2;
.....
else
    sentenciaN;
```

Idem con *bloques*



Estructuras de control selectivas

Ejemplo de uso selectiva múltiple

if (nota $\geq$ 9)

 resultado = "Sobresaliente";

else if (nota $\geq$ 7)

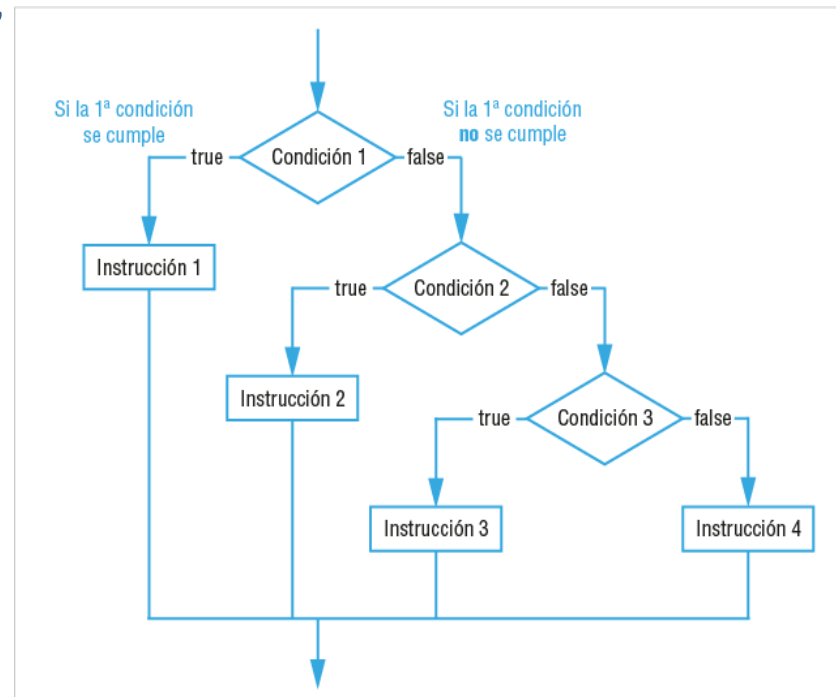
 resultado = "Notable";

else if (nota $\geq$ 5)

 resultado = "Aprobado";

else

 resultado = "Suspenso";



Se puede expresar con cuatro selectivas simples

Estructuras de control selectivas

Sintaxis de Anidamiento de Selectivas Doble

```
if (condición1) {  
    sentencia1;  
} else {  
    if (condición2) {  
        sentencia2;  
    } else {  
        sentencia3;  
    }  
}
```

Estructuras de control selectivas

Ejemplo de uso selectiva anidada

```
if (a != 0) {  
    x = -b / a;  
    resultado = "La solución es " + x;  
} else {  
    if (b != 0) {  
        resultado = "No tiene solución.";  
    } else {  
        resultado = "Solución indeterminada.";  
    }  
}
```

Se puede expresar con **tres selectivas simples**

Estructuras de control selectivas

```
1 package ejemplosClaseT1;
2
3 public class FacturaTaxi {
4     /**
5      * Programa que calcula la factura de un taxi segun el numero de kms.
6      * Por debajo de 30km la cantidad es fija, entre 31-100 km hay una segunda tarifa, y
7      * si recorre mas de 100km se aplica una tercera tarifa.
8      */
9
10    public static void main(String[] args) {
11        double fijo = 18;
12        int km = 150;
13        double tarifa2= 0.85;
14        double tarifa3= 0.65;
15        double importe;
16
17        if (km <= 30) {
18            importe = fijo;
19        } else if (km <= 100) {
20            importe = fijo + (km - 30) * tarifa2;
21        } else {
22            importe = fijo + 70 * tarifa2 + (km - 100) * tarifa3;
23        }
24        System.out.println();
25        System.out.println("CALCULO TARIFA TAXI");
26        System.out.println();
27        System.out.println("Kilometros recorridos => " + km);
28        System.out.printf("El importe total a pagar es %5.2f euros", importe);
29    }
30 }
31
```

Estructuras de control selectivas

Sintaxis del operador condicional ? : (Operador ternario)

Java proporciona una forma de abreviar una sentencia **if**:
el **operador condicional** (operador ternario) ? :

variable = condición ? expresión1 : expresión2;

equivale a

if (condición)

variable = expresión1;

else

variable = expresión2;

Estructuras de control selectivas

Sintaxis del operador condicional ? : (Operador ternario)

Java proporciona una forma de abreviar una sentencia **if**:
el **operador condicional** (operador ternario) ? :

variable = condición ? expresión1 : expresión2;

equivale a

if (condición)

variable = expresión1;

else

variable = expresión2;

Estructuras de control selectivas

Ejemplo de uso del operador condicional ? :

$\text{max} = (x > y) ? x : y;$ $\rightarrow$

```
if(x>y)
    max=x;
else
    max=y;
```

$\text{min} = (x < y) ? x : y;$ $\rightarrow$

```
if(x<y)
    min=x;
else
    min=y;
```

Estructuras de control selectivas

`max = (numero1 > numero2) ? numero1 : numero2;`

```
1 package ejemplosClaseT1;
2
3 import java.util.Scanner;
4
5 public class NumeroMaximo {
6
7     public static void main(String[] args) {
8
9         int numero1, numero2, max;
10
11         // Para leer desde teclado declaramos e inicializamos esta variable llamada entrada
12         Scanner entrada = new Scanner(System.in);
13
14         System.out.println("Introduzca primer numero: ");
15         numero1 = entrada.nextInt();
16
17         System.out.println("Introduzca segundo numero: ");
18         numero2 = entrada.nextInt();
19
20         if (numero1 > numero2)
21             max= numero1;
22         else
23             max= numero2;
24
25         System.out.println("El valor maximo es: " + max);
26
27     }
28
29 }
```



Estructuras de control selectivas

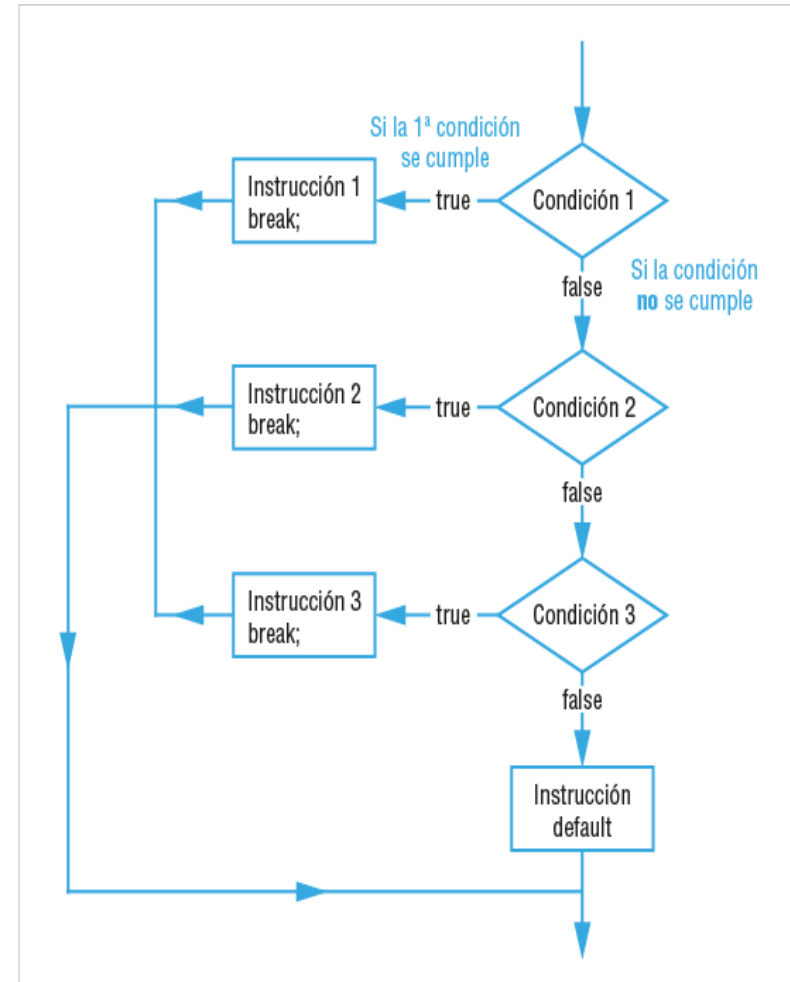
<i>absolute value</i>	<code>if (x < 0) x = -x;</code>
<i>put x and y into sorted order</i>	<pre> if (x > y) { int t = x; x = y; y = t; </pre>
<i>maximum of x and y</i>	<pre> if (x > y) max = x; else max = y; </pre>
<i>error check for division operation</i>	<pre> if (den == 0) System.out.println("Division by zero"); else System.out.println("Quotient = " + num/den); </pre>
<i>error check for quadratic formula</i>	<pre> double discriminant = b*b - 4.0*c; if (discriminant < 0.0) { System.out.println("No real roots"); } else { System.out.println((-b + Math.sqrt(discriminant))/2.0); System.out.println((-b - Math.sqrt(discriminant))/2.0); } </pre>

Typical examples of using if statements

Estructuras de control selectivas: `switch`

Sintaxis Selectiva Múltiple con la sentencia 'switch'

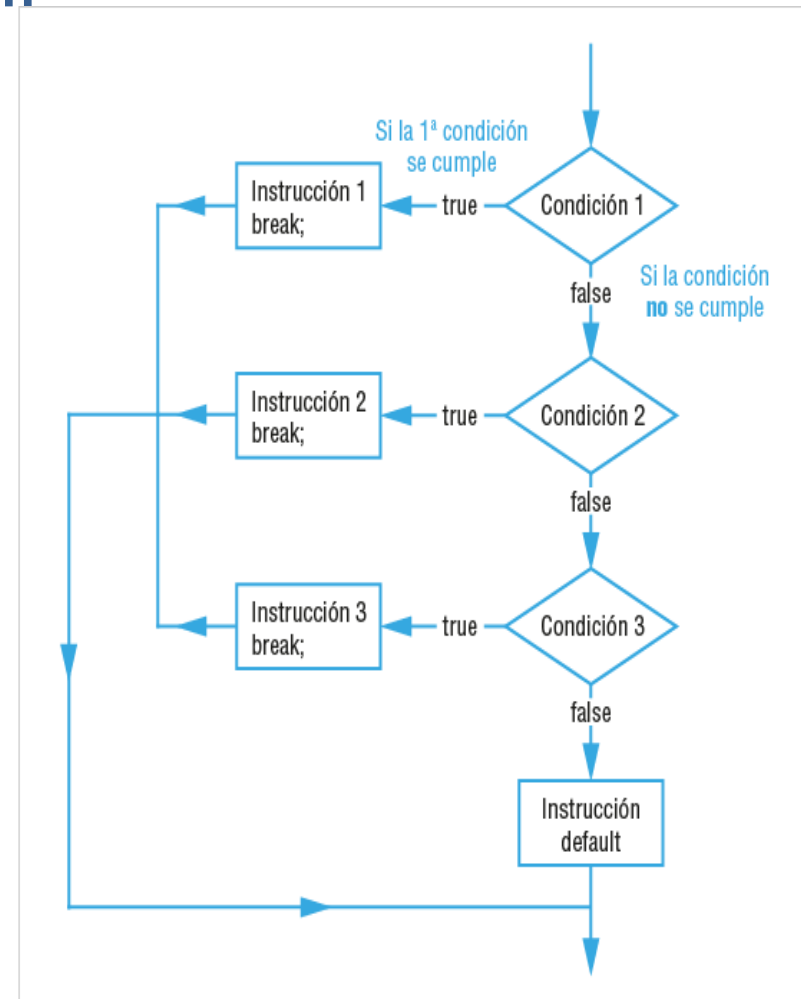
```
switch (expresión) {  
    case expr_cte1:  
        bloque1;  
        break;  
    case expr_cte2:  
        bloque2;  
        break;  
    ...  
    case expr_cteN:  
        bloqueN;  
        break;  
    default:  
        bloque_por_defecto;  
}
```



Estructuras de control selectivas: switch

Ejemplo Selectiva Múltiple con switch

```
int diaSemana= 4;  
switch (diaSemana) {  
case 1:  
    System.out.println("LUNES");  
    break;  
case 2:  
    System.out.println("MARTES");  
    break;  
case 3:  
    System.out.println("MIERCOLES");  
    break;  
case 4:  
    System.out.println("JUEVES");  
    break;  
case 5:  
    System.out.println("VIERNES");  
    break;  
case 6:  
    System.out.println("SABADO");  
    break;  
case 7:  
    System.out.println("DOMINGO");  
}
```

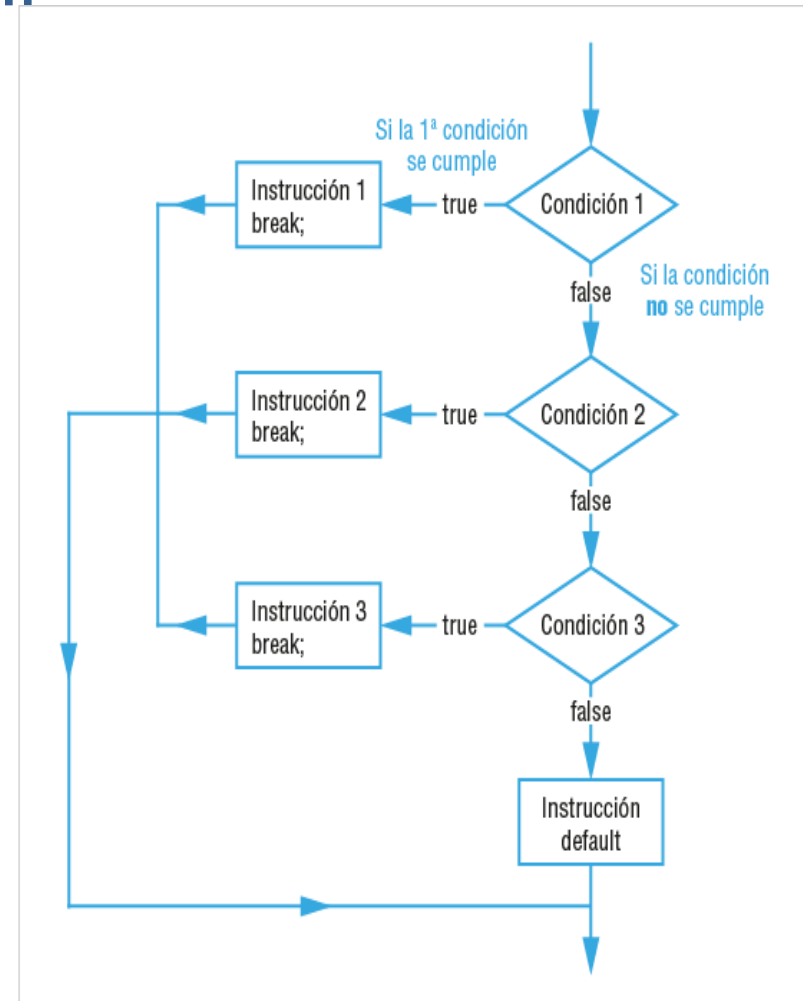


Estructuras de control selectivas: switch

Ejemplo Selectiva Múltiple con switch

```

int diaSemana= 4;
switch (diaSemana) {
→ case 1:
    System.out.println("LUNES");
    break;
→ case 2:
    System.out.println("MARTES");
    break;
→ case 3:
    System.out.println("MIERCOLES");
    break;
→ case 4:
    → System.out.println("JUEVES");
    break;
    case 5:
        System.out.println("VIERNES");
        break;
    case 6:
        System.out.println("SABADO");
        break;
    case 7:
        System.out.println("DOMINGO");
        break;
→ }
  
```



Estructuras de control selectivas: `switch`

Consideraciones al respecto

- Permite seleccionar entre varias alternativas posibles.
- Se selecciona a partir de la evaluación de una única expresión.
- La expresión del **`switch`** puede ser de tipo: **`byte`, `short`, `char`, `int`, `String`, ...**
- Los valores de cada caso del **`switch`** han de ser **constantes**.
- En Java, cada bloque de código de los que acompañan a un posible valor de la expresión entera ha de terminar con una sentencia **`break`**;
- La etiqueta **`default`** marca el bloque de código que se ejecuta por defecto (cuando al evaluar la expresión se obtiene un valor no especificado por los casos anteriores del **`switch`**). Es opcional.
- En Java, se pueden poner varias etiquetas seguidas acompañando a un único fragmento de código si el fragmento de código que ha de ejecutarse es el mismo para varios valores de la expresión entera que gobierna la ejecución del **`switch`**.

Estructuras de control selectivas: switch



Estructuras de control selectivas: switch

```
switch (nota) {  
    case 0:  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
        resultado = "Suspenso";  
        break;  
    case 5:  
    case 6:  
        resultado = "Aprobado";  
        break;  
    case 7:  
    case 8:  
        resultado = "Notable";  
        break;  
    case 9:  
    case 10:  
        resultado = "Sobresaliente";  
        break;  
    default:  
        resultado = "Error";  
}
```

// CÓDIGO EQUIVALENTE

```
int nota=3;  
String resultado;  
if ( nota >= 9 )  
    resultado = "Sobresaliente";  
else if ( nota >= 7 )  
    resultado = "Notable";  
else if ( nota >= 5 )  
    resultado = "Aprobado";  
else if ( nota >= 0 )  
    resultado = "Suspenso";  
else  
    resultado = "Error";  
  
System.out.println("La calificacion es "+resultado);
```

```
System.out.println("La calificacion es: " + resultado);
```

Estructuras de control selectivas: switch

```
int numeroDia=0;
String dia = "jueves";
switch (dia) {
    case "lunes":
        numeroDia=1;
        break;
    case "martes":
        numeroDia=2;
        break;
    case "miercoles":
        numeroDia=3;
        break;
    case "jueves":
        numeroDia=4;
        break;
    case "viernes":
        numeroDia=5;
        break;
    case "sabado":
        numeroDia=6;
        break;
    case "domingo":
        numeroDia=7;
        break;
    default:
        numeroDia=0;
}
System.out.print("El numero del dia '"+ dia + "' es: " + numeroDia);
```

/ Dado un nombre de día de la semana "lunes", "martes, etc,
muestre el número día al que corresponde entre el 1 y el 7 */*

Estructuras de control selectivas: switch

```
public class MuestraFecha
```

```
// MENU
```

```
int opcion = 0;
System.out.println("Formatos de fecha: ");
System.out.println("-----");
System.out.println("1. Formato corto (europeo): DD/MM/AAAA");
System.out.println("2. Formato corto (americano): MM/DD/AAAA");
System.out.println("3. Formato largo: DD de MMMM de AAAA");
System.out.print("Elija una opción: ");
opcion = entrada.nextInt();
```

```
switch (opcion) {
    case 1:
        ...
        break;
    case 2:
        ...
        break;
    case 3:
        ...
        break;
    default:
        ...
}
```

```
// MENU
```

Formatos de fecha:

1. Formato corto (europeo): DD/MM/AAAA
 2. Formato corto (americano): MM/DD/AAAA
 3. Formato largo: DD de MMMM de AAAA
- Elija una opción:

Tema 1. Fundamentos de Programación



Estructuras de control

- ☐ Programación estructurada
- ☐ Estructuras selectivas/condicionales
-  ☐ Estructuras repetitivas/iterativas

Tema 1. Fundamentos de Programación

➤ Estructuras de control repetitivas / iterativas / bucles

Permiten **repetir una acción o conjunto de acciones** un determinado número de veces.

En ocasiones, ese número es conocido de antemano, pero en otras ocasiones, no.

Tema 1. Fundamentos de Programación

➤ Estructuras de control repetitivas / iterativas / bucles

En un bucle hay que tener en cuenta dos aspectos:

➤ La condición de terminación,

es una **expresión booleana** que indica cuándo se deja de ejecutar el bucle. Coloquialmente se dice que es la condición que me permite escapar o salir del bucle.

➤ Cuerpo del bucle,

es la acción o conjunto de acciones que se repiten.

Estructuras de control repetitivas

- En un bucle se **repite** la ejecución de una sentencia o un bloque de sentencias siempre que la **condición de terminación sea verdadera (true)**
 - Cada repetición de todas las instrucciones del cuerpo del bucle es una **iteración**.
- **Al final de cada iteración** se vuelve a evaluar la condición, y de ser **verdadera (true)**, se ejecuta una nueva iteración.
- En el momento en el que la **condición sea falsa (false)**, se dejan de ejecutar las sentencias y **se escapa o sale** del bucle.

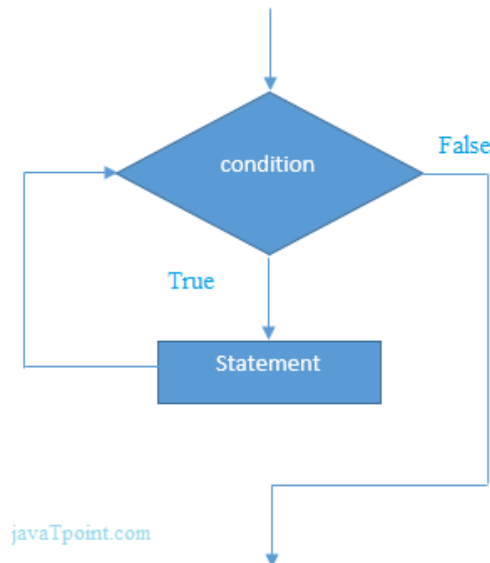
Estructuras de control repetitivas

- Tres tipos de bucles:
 - Bucle **while**
 - Bucle **for**
 - Bucle **do-while**

Estructuras de control repetitivas

El bucle while

- La condición de terminación se comprueba **antes** de comenzar la ejecución de las instrucciones. **Si es falsa** (`false`) NO hace nada.
- **Si es verdadera** (`true`) ejecuta todas las sentencias del cuerpo del bucle y cuando las termina vuelve a comprobar la condición.
- Se repiten los pasos hasta que la condición sea **falsa** (`false`).



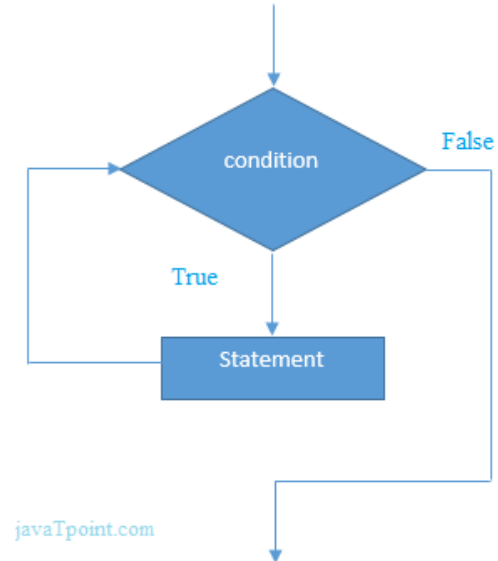
Estructuras de control repetitivas

El bucle while

- **Sintaxis:**

while (condición de continuación del bucle)
 sentencia; // Cuerpo del bucle

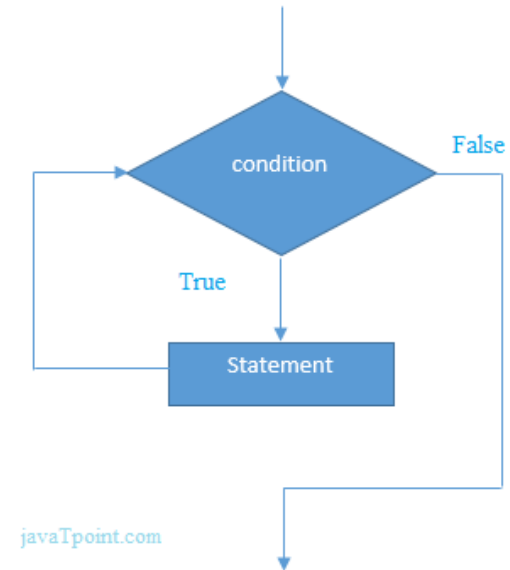
```
while (condición) {  
    bloque  
}
```



Estructuras de control repetitivas

*El bucle **while***

- El comportamiento del bucle **while**:

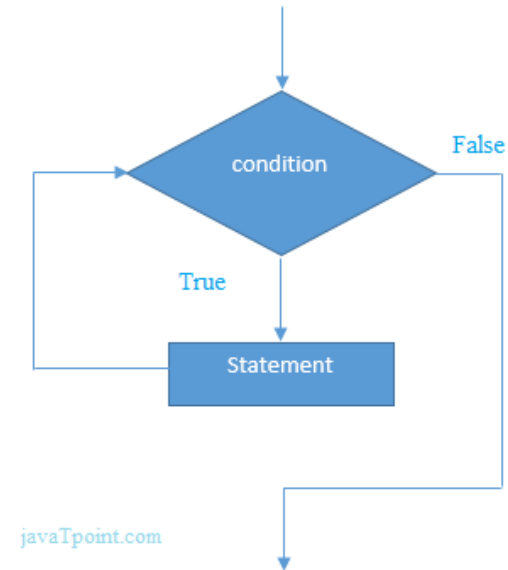


1. Se evalúa la **condición** (condición de terminación) asociada al bucle.
2. Si la evaluación de la condición es **false**, termina la ejecución del bucle.
3. Si por el contrario, la evaluación de la condición es **true**, se ejecuta el bloque de instrucciones
4. Tras ejecutar el bloque de instrucciones, se vuelve al punto 1.

Estructuras de control repetitivas

El bucle *while*

- El comportamiento del bucle **while**:



Un bucle **while** puede realizar cualquier número de iteraciones, desde **cero**, cuando la primera evaluación de la condición resulta **false**.

En el caso de que la condición siempre sea **true** y las variables que la controlan nunca tomen valores que la hagan **false**, tendremos lo que se conoce como **bucle infinito**

Estructuras de control repetitivas

Ejemplo. Ejemplo de un fragmento de código donde se utiliza una estructura repetitiva **while**

.....

```
int i = 1;
```

```
while (i <= 5) {
```

```
    System.out.println("Iteración número " + i);
```

```
    i++;
```

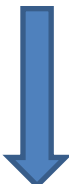
```
}
```

.....

Estructuras de control repetitivas

Seguimiento Manual

```
int i = 1;
while (i <= 5) {
    System.out.println("Iteración número "+i);
    i++;
}
```



Iteración	i	Salida a pantalla
0	1	-----
1	2	Iteración número...1
2	3	Iteración número...2
⋮	⋮	⋮
5	6	Iteración número...5

Estructuras de control repetitivas

Ejemplo.

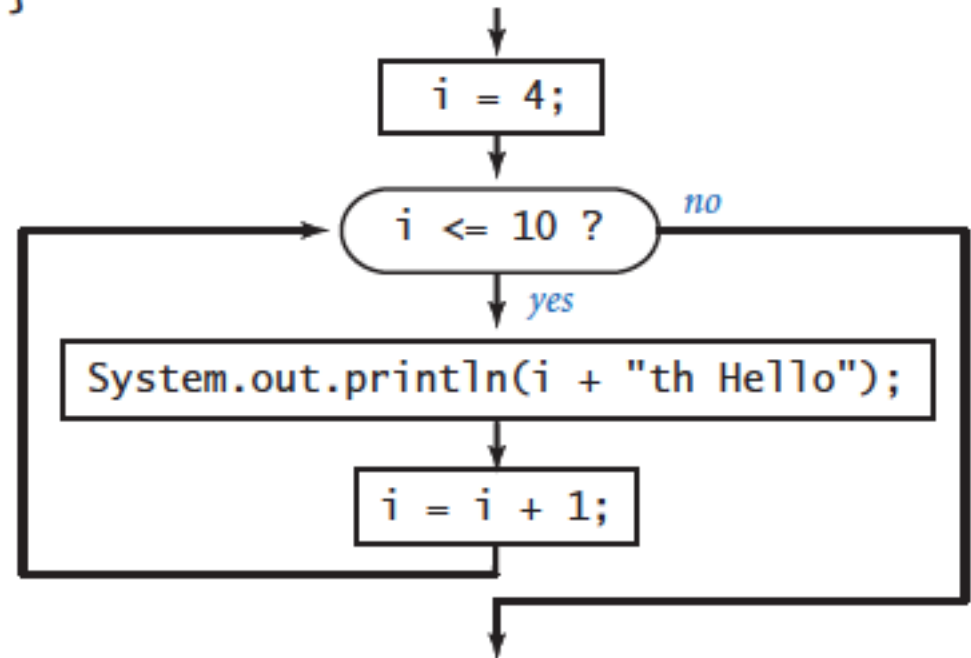
Estructura repetitiva **while**

-Este ciclo se repite 7 veces:
para $i = 4, 5, 6, 7, 8, 9$ y 10

-En la última iteración ($i=10$) se incrementa el valor de i y pasa a valer $i=11$.

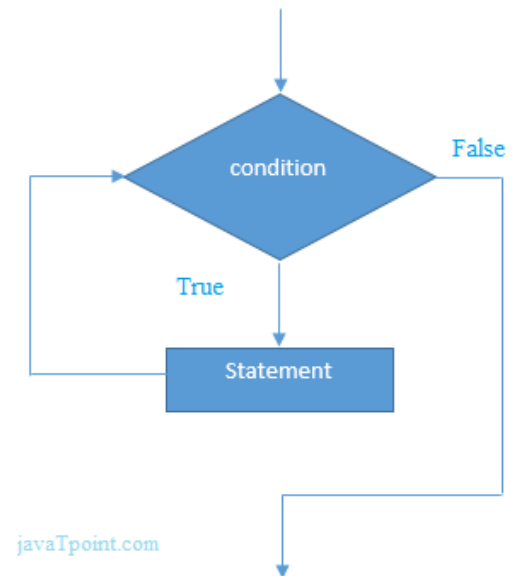
-> La expresión se evalúa a false

```
int i = 4;  
while (i <= 10)  
{  
    System.out.println(i + "th Hello");  
    i = i + 1;  
}
```



Estructuras de control repetitivas

- La ventaja que tiene esta estructura (**while**) es que, comprobada la condición por primera vez, si es `false`, **nunca se llega a ejecutar** el bucle.
- También cabe la posibilidad de que **si la condición no está bien construida**, nunca se haga `false` con lo cual tendríamos los *temidos bucles infinitos*.



Estructuras de control repetitivas

Ejemplo. Bucle que no se ejecuta nunca

```
.....  
int i = 7;  
while (i <= 5) {  
    System.out.println("Iteración número " + i);  
    i++;  
}  
.....
```

Estructuras de control repetitivas

Ejemplo. Bucle que no se ejecuta nunca

```
.....  
int i = 7;  
while (i <= 5) {  
    System.out.println("Iteración número " + i);  
    i++;  
}  
.....
```



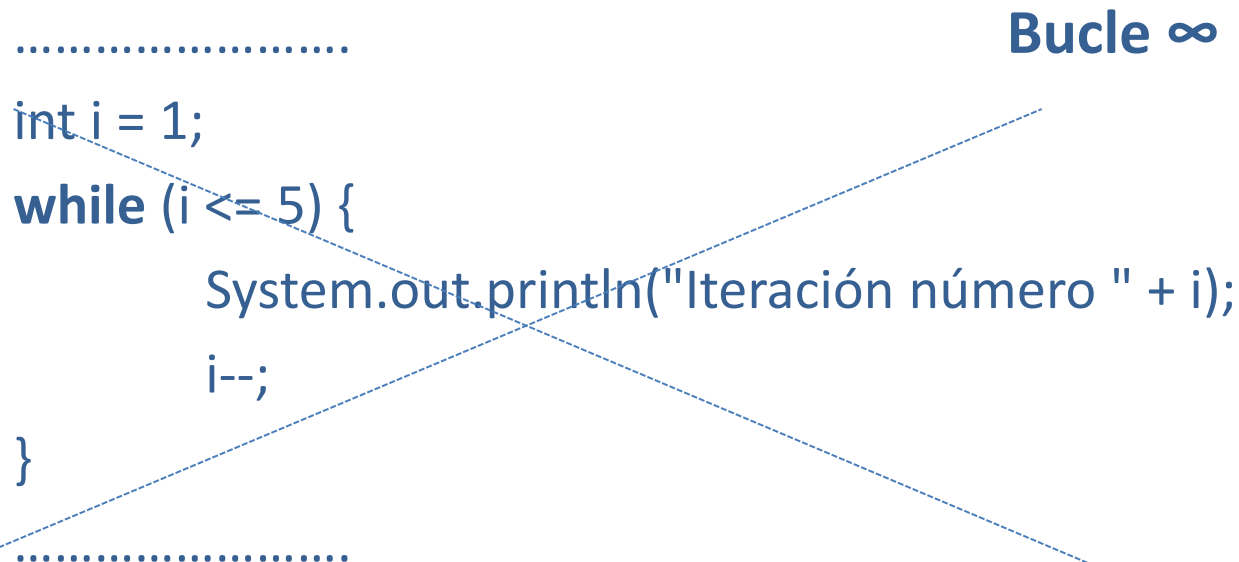
Estructuras de control repetitivas

Ejemplo. Bucle infinito

.....

Bucle ∞

```
int i = 1;  
while (i <= 5) {  
    System.out.println("Iteración número " + i);  
    i--;  
}  
.....
```



Estructuras de control repetitivas

Ejemplo. Bucle infinito

.....

int i = 1;

while (i <= 5) {

System.out.println("Iteración número " + i);

i--;

}

.....

Bucle ∞

Estructuras de control repetitivas

En el bucle son característicos dos elementos: **contadores** y **sumadores**.
A continuación, se describen con un ejemplo.

```
.....  
int i = 1;  
suma = 0;  
dato = 2;  
while (i <= 5) {  
    suma = suma + dato;           // suma += dato;  
    dato = dato + 2;             // dato += 2;  
    i = i + 1;                   // i++;  
}  
.....
```

Contador: variable *i* que se incrementa/decrementa en una cantidad fija en cada iteración o paso.

Sumador: variable *suma* que se incrementa/decrementa en una cantidad variable en cada iteración o paso.

Estructuras de control repetitivas

En el bucle son característicos dos elementos: **contadores** y **sumadores**.

```
1 package ejemplosClaseT1;
2
3 public class ejemploWhile {
4     public static void main(String[] args) {
5         int sum = 0;
6         int number = 0;
7         while (number < 20) {
8             number++;
9             sum += number;
10            System.out.println("Iteracion = " + number);
11        }
12
13        System.out.println();
14        System.out.println("El numero es " + number);
15        System.out.println("La suma vale " + sum);
16    }
17 }
```

Contador: variable *i* que se incrementa/decrementa en una cantidad fija en cada iteración o paso.

Sumador: variable *suma* que se incrementa/decrementa en una cantidad variable en cada iteración o paso.

Estructuras de control repetitivas

Salidas anticipadas de bucles - Terminar bucle con anterioridad

- **break** finaliza completamente el bucle, sale del bucle
- **continue** detiene la iteración actual, y continua con la siguiente
- Cualquier programa puede escribirse sin **break** y **continue**, incluso hay autores que recomiendan no utilizarlos porque rompen la secuencia natural de ejecución de las instrucciones

```
i = 1;
while (i <= 10) {
    System.out.println("La i vale " + i);
    if (i == 2) {
        break;
    }
    i++;
}

i = 0;
while (i < 10) {
    i++;
    if (i % 2 == 0) {
        continue;
    }
    System.out.println("La i vale " + i);
}
```

Estructuras de control repetitivas

Salidas anticipadas de bucles - Terminar bucle con anterioridad

- **break** finaliza completamente el bucle, sale del bucle

```
1 package ejemplosClaseT1;
2
3 public class ejemBreak {
4     public static void main(String[] args) {
5         int sum = 0;
6         int number = 0;
7         while (number < 30) {
8             number++;
9             sum += number;
10            System.out.println("Iteracion = " + number);
11            if (sum >= 100)
12                break; //se sale del bucle y pasa a la linea 14
13        }
14
15        System.out.println("El numero es " + number);
16        System.out.println("La suma vale " + sum);
17    }
18 }
19
```

Estructuras de control repetitivas

Salidas anticipadas de bucles - Terminar bucle con anterioridad

- **continue** detiene la iteración actual, y continua con la siguiente

```
1 package ejemplosClaseT1;
2
3 public class ejemContinue {
4     public static void main(String[] args) {
5         int sum = 0;
6         int number = 0;
7         while (number < 20) {
8             number++;
9             System.out.println("Iteracion = " + number);
10            if (number == 10 || number == 11)
11                continue; //Pasa a la línea 7
12
13            sum += number;
14        }
15
16        System.out.println("El numero es " + number);
17        System.out.println("La suma vale " + sum);
18    }
19 }
```

Estructuras de control repetitivas

Ejemplo

- web

```
1 package ejemplosClaseT1;
2 import java.util.Scanner;
3
4 public class liangGuessNumber {
5     public static void main(String[] args) {
6         // Generate a random number to be guessed
7         int number = (int)(Math.random() * 101);
8
9         Scanner input = new Scanner(System.in);
10        System.out.println("Guess a magic number between 0 and 100");
11
12        int guess = -1;
13        while (guess != number) {
14            // Prompt the user to guess the number
15            System.out.print("\nEnter your guess: ");
16            guess = input.nextInt();
17
18            if (guess == number)
19                System.out.println("Yes, the number is " + number);
20            else if (guess > number)
21                System.out.println("Your guess is too high");
22            else
23                System.out.println("Your guess is too low");
24        } // End of loop
25    }
26 }
```

Estructuras de control repetitivas

- Tres tipos de bucles:
 - Bucle **while**
 - Bucle **for**
 - Bucle **do-while**

Estructuras de control repetitivas

El bucle for

- Ayuda a simplificar la escritura de un bucle especialmente cuando conocemos de antemano el **número de iteraciones** del bucle.

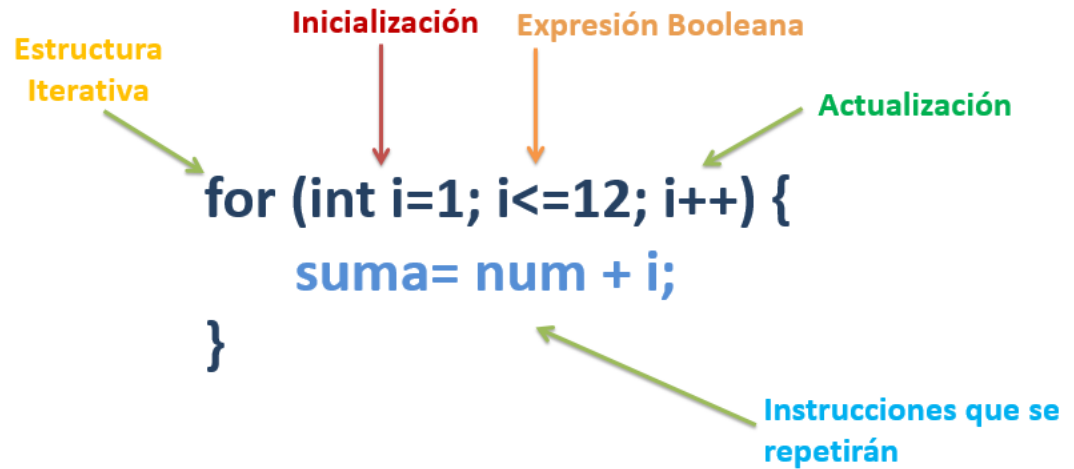
- **Sintaxis**

```
for (expr1; expr2; expr3) {  
    bloque;    // Cuerpo del bucle  
}
```

- **Donde**

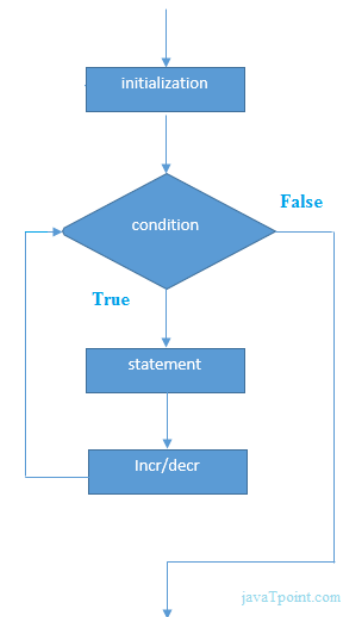
- **expr1**: Inicialización
- **expr2**: Condición de terminación (expresión booleana)
- **expr3**: Incremento

Estructuras de control



El bucle *for*

1. Se ejecuta la **inicialización** (de uno o más variables) y se hace una sola vez, al principio.
2. Después se evalúa la **condición de terminación**, si es `false`, salimos del bucle y continuamos con el resto del programa; en caso de que la evaluación sea `true`, se ejecuta todo el bloque de instrucciones.
3. Cuando termina de ejecutarse el bloque de instrucciones se ejecuta el **incremento**.
4. Se vuelve de nuevo a evaluar la **condición de terminación** del bucle (volver al punto 2).



Estructuras de control repetitivas

Ejemplo. Ejemplo de un fragmento de código donde se utiliza una estructura repetitiva o bucle **for**

```
.....  
for (int i = 1; i <= 5; i++) {  
    System.out.println("Iteración número " + i);  
}  
.....
```

Salida

```
Iteración número 1  
Iteración número 2  
Iteración número 3  
Iteración número 4  
Iteración número 5
```

Estructuras de control repetitivas

Ejemplo. Ejemplo de un fragmento de código donde se utiliza una estructura repetitiva **for**

initialize another variable in a separate statement → `int v = 1;`

declare and initialize a loop control variable → `int i = 0;`

loop continuation condition → `i <= N;`

increment → `i++`

```
for (int i = 0; i <= N; i++)  
{  
    System.out.println(i + " " + v);  
    v = 2*v;  
}
```

body ↑

Anatomy of a for loop (that prints powers of 2)

Estructuras de control repetitivas

- Esta estructura está especialmente indicada para **recorridos de arrays** y **matrices** que veremos más adelante.
- Respecto al **funcionamiento** de la estructura **for**, está gobernada por una variable de control que se inicializa a un valor inicial (**expr1**) y dicha variable se va **incrementando o decrementando** en cada paso o iteración en una cantidad fija que se indica en **expr3** hasta que la variable alcanza el valor final que se expresa en **expr2** .

```
for (int i = 5; i <= 15; i++) {  
    System.out.println("Iteración número " + i);  
}
```

Estructuras de control repetitivas

- Existe también la posibilidad de que las sentencias que componen el cuerpo del bucle **no se ejecuten ninguna vez**. Por ejemplo:

```
for (int i = 7; i <= 5; i++) {  
    System.out.println("Iteración número " + i);  
}
```

Estructuras de control repetitivas

Bucles con **decremento** de la variable de control en cada iteración

```
int n = 5;  
for (int i = n; i >= 1; i--) {  
    System.out.println("Iteración número " + i);  
}
```

Salida

Iteración número 5
Iteración número 4
Iteración número 3
Iteración número 2
Iteración número 1

Estructuras de control repetitivas

Equivalencia entre for y while

Un fragmento de código como el que sigue con un bucle **while**:

```
int i = 0;
int n = 10;
while (i <= 10) {
    System.out.println (n + " x " + i + " = " + (n * i));
    i++;
}
```

puede **abreviarse** si utilizamos un bucle **for**:

```
int n = 10;
for (int i = 0; i <= 10; i++) {
    System.out.println (n + " x " + i + " = " + (n * i));
}
```

Estructuras de control repetitivas

Equivalencia entre for y while

```
1 package ejemplosClaseT1;
2
3 public class ejemploWhile {
4     public static void main(String[] args) {
5         int sum = 0;
6         int number = 0;
7         while (number < 20) {
8             number++;
9             sum += number;
10            System.out.println("Iteracion = " + number);
11        }
12
13        System.out.println();
14        System.out.println("El numero es " + number);
15        System.out.println("La suma vale " + sum);
16    }
17 }
```

Contador: variable i

Sumador: variable suma

Estructuras de control repetitivas

Equivalencia entre for y while

```
1 package ejemplosClaseT1;
2
3 public class ejemploFor {
4     public static void main(String[] args) {
5         int sum = 0;
6         int number = 0;
7         for (int i=0; i<20; i++){
8             number++;
9             sum += number;
10            System.out.println("Iteracion = " +i); //podemos poner i y number
11        }
12
13        System.out.println();
14        System.out.println("El numero es " + number); //NO podemos poner i
15        System.out.println("La suma vale " + sum);
16    }
17 }
```

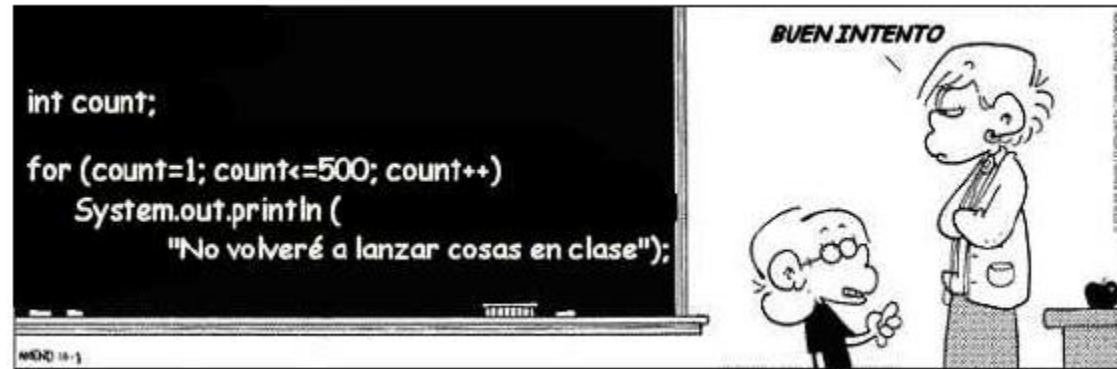
Estructuras de control repetitivas

En general,

```
for (expr1; expr2; expr3) {  
    bloque;  
}
```

equivale a

```
expr1;  
while (expr2) {  
    bloque;  
    expr3;  
}
```



Estructuras de control repetitivas

Equivalencia entre for y while

Un fragmento de código como el que sigue con un bucle **while**:

```
while (dato != 0) {  
    // Cuerpo del bucle  
}
```

```
while (condición de continuidad) {  
    // Cuerpo del bucle  
}
```

puede expresarse si utilizamos un bucle **for**:

```
for (; dato != 0; ) {  
    // Cuerpo del bucle  
}
```

```
for (; condición de continuidad; ) {  
    // Cuerpo del bucle  
}
```

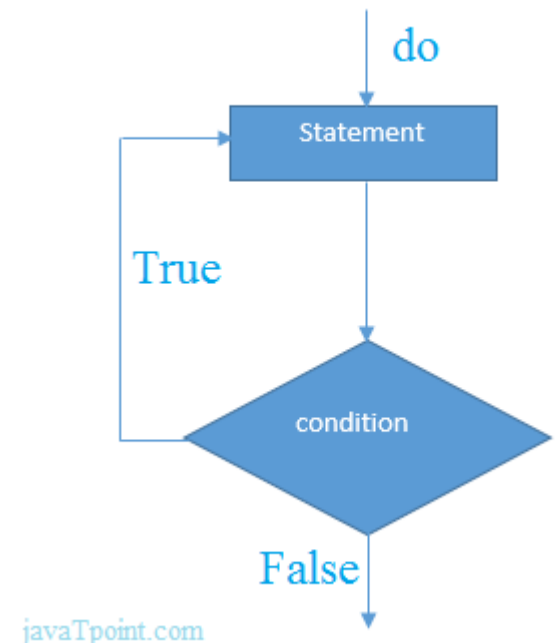
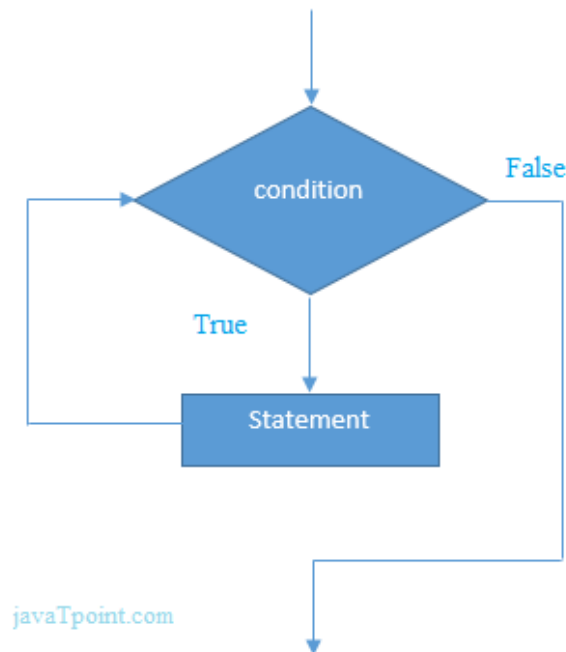
Estructuras de control repetitivas

- Tres tipos de bucles:
 - Bucle **while**
 - Bucle **for**
 - Bucle **do-while**

Tema 1. Fundamentos de Programación

*El bucle **do-while***

Es un tipo de bucle similar al **while**, que realiza la comprobación de la condición **después** de ejecutar el cuerpo del bucle.



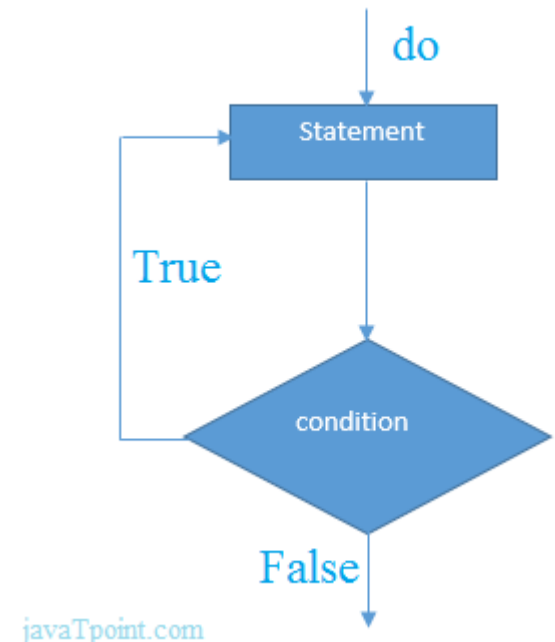
Tema 1. Fundamentos de Programación

*El bucle **do-while***

Es un tipo de bucle similar al **while**, que realiza la comprobación de la condición **después** de ejecutar el cuerpo del bucle.

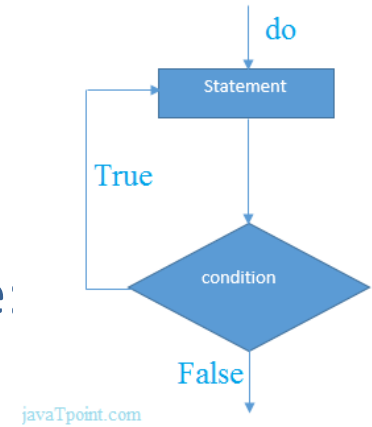
Sintaxis

```
do  
    sentencia;  
while (condición);  
  
do {  
    bloque  
} while (condición);
```



Estructuras de control repetitivas

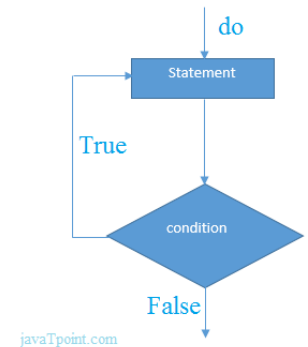
- El funcionamiento del bucle **do-while** es el siguiente:
 1. Se ejecuta el bloque de instrucciones.
 2. Se evalúa la condición.
 3. Según el valor obtenido en la **condición** se termina el bucle si es **false** o se comienza en el punto 1 si es **true**.
- Debemos recordar que **do-while** es la única estructura de bucle que finaliza en punto y coma (;).
- Mientras que el bucle **while** se puede ejecutar de 0 a infinitas veces, el bucle **do-while** lo hace **de 1** a infinitas veces.
- De hecho, la única diferencia con el bucle **while** es que el cuerpo del bucle **do-while** se ejecuta, al menos una vez.



Estructuras de control repetitivas

- El bloque de instrucciones se ejecuta **al menos una vez**.
- El bucle **do-while** resulta especialmente indicado para **validar datos** de entrada.
- Por ejemplo, **comprobar** que los valores de entrada obtenidos están dentro del rango de valores que el programa espera:

```
do {  
    System.out.print("Introduce un valor numérico para el mes: ");  
    mes = entrada.nextInt();  
} while (mes < 1 || mes > 12);
```



```
1 package org.ip.tema01.iterativas;
2
3 import java.util.Scanner;
4
5 public class PruebaDoWhile {
6
7     public static void main(String[] args) {
8         int dato;
9         int suma = 0;
10
11         // Crear un scanner para leer datos de entrada
12         Scanner entrada = new Scanner(System.in);
13
14         // Continuar leyendo hasta que la entrada sea 0
15         do {
16             // Leer los datos siguientes
17             System.out.print("Introduce un entero (la entrada termina con 0): ");
18             dato = entrada.nextInt();
19             suma += dato;
20         } while (dato != 0);
21
22         System.out.println("La suma es: "+ suma);
23         entrada.close();
24     }
25 }
```

```
Introduce un entero (la entrada termina con 0): 1
Introduce un entero (la entrada termina con 0): 3
Introduce un entero (la entrada termina con 0): 5
Introduce un entero (la entrada termina con 0): 0
La suma es 9
```

```
1 package org.ip.tema01.iterativas;
2
3 import java.util.Scanner;
4
5 public class PruebaWhile {
6
7     public static void main(String[] args) {
8         int dato;
9         int suma = 0;
10
11         // Crear un scanner para leer datos de entrada
12         Scanner entrada = new Scanner(System.in);
13         System.out.print("Introduce un entero (la entrada termina con 0): ");
14         dato = entrada.nextInt();
15
16         // Continuar leyendo hasta que la entrada sea 0
17         while (dato != 0){
18             suma += dato;
19             // Leer los datos siguientes
20             System.out.print("Introduce un entero (la entrada termina con 0): ");
21             dato = entrada.nextInt();
22         }
23
24         System.out.println("La suma es: " + suma);
25         entrada.close();
26     }
27 }
```

```
Introduce un entero (la entrada termina con 0): 1
Introduce un entero (la entrada termina con 0): 3
Introduce un entero (la entrada termina con 0): 5
Introduce un entero (la entrada termina con 0): 0
La suma es 9
```

¿Qué bucle utilizar?

- Los bucles **while** y **for** se llaman bucles *pre-test* porque la condición de continuación se comprueba antes de ejecutar el bucle.
- El bucle **do-while** se llama *post-test* porque la condición se comprueba después de que el bucle se ha ejecutado.
- Las tres sentencias de bucles: **for**, **while** y **do-while** son EQUIVALENTES es decir podemos escribir un bucle con cualquiera de ellas, con la que estemos más cómodos, pero lo habitual es:
 - Si se conocen de antemano el número de iteraciones => **for**
 - Si no se conoce dicho número de iteraciones => **while**
 - Para validar datos => **do-while**

Estructuras de control repetitivas

Bucles anidados

Se trata de uno o varios bucles **contenidos** en otros, es decir, tendremos un bucle cuyo cuerpo contenga al menos una sentencia que sea a su vez otro bucle y éste a su vez puede tener otra sentencia que sea un bucle.

Por ejemplo:

```
int N = 3;
for (int i = 0; i < N; i++) {           // Bucle externo
    for (int j = 0; j < N; j++) {       // Bucle interno
        System.out.print("(" + i + "," + j + ") " + '\t');
    }
    System.out.println();
}
```

Salida	(0,0)	(0,1)	(0,2)
	(1,0)	(1,1)	(1,2)
	(2,0)	(2,1)	(2,2)

Estructuras de control repetitivas

Bucles anidados

- En el uso de bucles es muy frecuente la anidación. Al hacer esto se **multiplica** el número de veces que se ejecuta el bloque de instrucciones de los bucles internos.
- Los bucles anidados pueden encontrarse relacionados (**dependientes**) cuando los valores de las variables de los bucles más externos intervienen en el control de la iteración de un bucle interno de un bucle interno, o
- **independientes**, cuando no existe relación alguna entre ellos, siendo el número de iteraciones de un bucle ajeno a los valores de las variables utilizadas en otro bucle, ya sea externo o interno.

Anidar bucles es una herramienta que facilita el procesamiento de **tablas multidimensionales**, utilizándose cada nivel de anidación para manejar los índices de la dimensión correspondiente.

Estructuras de control repetitivas

Bucles anidados independientes

Cuando los bucles anidados **no dependen** unos de los otros para determinar el número de iteraciones se denominan, **bucles anidados independientes**.

```
for (int i = 1; (i <= 4); i++) {  
    System.out.println("+ " + i);  
    for (int j = 1; (j <= 3); j++) {  
        System.out.println(".. * " + j);  
    }  
}  
  
for (int i = 1; (i <= 2); i++) {  
    System.out.println("+ i = " + i);  
    for (int j = 1; (j <= 3); j++) {  
        System.out.println(".. * j = " + j);  
        for (int k = 1; (k <= 4); k++) {  
            System.out.println(".... - k = " + k);  
        }  
    }  
}
```

Estructuras de control repetitivas

Bucles anidados dependientes

Puede darse el caso que el número de iteraciones de un bucle **no sea independiente** de la ejecución de los bucles externos, y dependa de sus variables de control, con lo que nos encontramos con los denominados **bucles anidado dependientes**.

En este ejemplo la variable *i* se toma como base para comparar con los valores de la variable *j* que controla el bucle más interno.

```
for (int i = 1; (i <= 4); i++) {  
    System.out.println("+ i = " + i);  
    int j = 1;  
    while (j <= i) {  
        System.out.println(".. * j = " + j);  
        j++;  
    }  
}
```

Estructuras de control repetitivas

Diseño de un bucle

A partir del enunciado de un problema, ¿cómo puedo construir correctamente un bucle?. En primer lugar, optaremos por una estructura repetitiva (bucle) siempre que haya proceso que se **repite**. A continuación, puede ser de ayuda responder a las preguntas:

- ¿Cuál es la condición de terminación del bucle?
- ¿Cómo se inicializa y actualiza la condición?
- ¿Cuál es el proceso que se repite?
- ¿Cómo se inicializa y actualiza el proceso?

Tabla de Multiplicar (libro Liang)

```

1 package ejemplosClaseT1;
2
3 public class liangMultiplicationTable { //chap 5
4     /** Main method */
5     public static void main(String[] args) {
6         // Display the table heading
7         System.out.println("          Multiplication Table");
8
9         // Display the number title
10        System.out.print("          ");
11        for (int j = 1; j <= 9; j++)
12            System.out.print("    " + j);
13
14        System.out.println("\n-----");
15
16        // Print table body
17        for (int i = 1; i <= 9; i++) {
18            System.out.print(i + " | ");
19            for (int j = 1; j <= 9; j++) {
20                // Display the product and align properly
21                System.out.printf("%4d", i * j);
22            }
23            System.out.println();
24        }
25    }
26 }

```

	Multiplication Table								
	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36
5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81

¡MUCHAS GRACIAS!



UNIVERSIDAD DE ALMERÍA

